Chapter 1

AI Engineering Is Engineering

1. The Engineering Question

Most AI projects start with a technology looking for a place to go. A team gets access to a capable model, someone asks where it could be used, and within a week there is a list of features that could “add AI”. Some of those features turn out well. Many turn out to be expensive ways of doing something a simpler component already did, or would have done, at lower cost and with fewer surprises.

The question behind that list is:

> “Where can we use an LLM?”

It sounds practical and it is the wrong starting point, for the same reason “where can we use a message queue?” is the wrong starting point. It begins with the solution and goes looking for problems that fit it.

This book starts from a different question:

What problem are we trying to solve, and what is the simplest technology that solves it well enough?

That question is not an argument against AI. Sometimes the answer is a language model, and sometimes nothing simpler will do. The point is that the answer comes *after* the problem has been stated, the constraints have been named and the alternatives have been compared. Starting from the problem does not rule AI out. It puts AI in the same position as every other technology: a choice that has to earn its place, not a starting assumption.

Language models make this discipline harder to keep, for reasons that will come up again and again in this book:

- **They are easy to call.** Adding a model call often takes a few lines of code. The ease of the call hides the cost of everything around it: latency, model management, evaluation, failure handling.
- **Their output is persuasive.** A fluent answer looks correct. A demo that works on three examples looks like a feature.
- **They are learned from data rather than fully specified by written rules.** Depending on the model and inference setup, outputs may be deterministic or stochastic; similar inputs can still produce different results, and the cause may not be visible in the code.
- **They are partly observable.** You can see what went in and what came out. You usually cannot see why one output was produced instead of another.

None of these properties makes AI engineering a different discipline. They make the ordinary discipline of engineering — state the problem, know what must be true, gather evidence, compare alternatives, decide, and know when to revisit — more necessary, not less.

This chapter's case study is small and real. A simple keyword rule decided whether each request to one of the lab system's entry points should get an answer or a diagram. That design was deliberate, and its weakness was on record. Then a user attached an image and asked what was in it, and the rule sent the request down the wrong path. The fix replaced the rule with a model call, which added one model call to every request.

The case is not in this chapter because the model won. It is here because it shows the questions this book is about: when a deterministic rule stops being sufficient for a problem, what the alternative costs, and what you know and don't know after making the change.

By the end of the chapter you should be able to:

- state a problem before choosing a technology, and say what "well enough" means for it;
- tell whether a part of your system needs to *match* or to *interpret*;
- list the costs a learned-model component adds beyond the call itself;
- describe the engineering loop the rest of the book applies, and say which of its stages a given decision actually went through.



2. Core Concepts

Problem-first engineering

A problem statement is the most underrated artifact in an AI project. It answers four questions before any technology is mentioned:

- **What is the task?** What should the system do, for whom, in which situations?
- **What does "well enough" mean?** What quality, latency and cost would make the result acceptable? What failure would make it unacceptable?
- **What must always be true?** Which constraints are not negotiable: who may see which data, which actions are allowed, what must never be sent outside the system?
- **How will we know?** What observation or measurement would tell us the problem is solved?

A team that can't answer these questions isn't ready to choose a technology. It can still prototype, and prototypes are useful for learning what the problem is. But a prototype that "works" is evidence about the prototype, not about whether the problem has been solved.

> *Illustrative.* "Add an AI assistant to the support page" is a solution. "Reduce the number of support tickets that ask for information already in the documentation, without giving users wrong answers about billing" is a problem. Only the second tells you what to measure, what to protect and which alternatives to compare. A better search box, a rewritten documentation page and a language model are all candidates for it.

Problem-first engineering does not mean the problem statement is fixed. Problems change, and the case study in this chapter is about exactly that. When the problem changes, re-examine the technology against the new problem instead of patching the old solution.

The simplest technology that solves the problem well enough

The principle this book returns to is:

Use the simplest technology that solves the problem well enough.

Three words in it carry the weight.

Simplest does not mean "non-AI". It means the option with the least added cost, complexity and uncertainty *for this problem*. Sometimes that is a lookup table or a search index. Sometimes it is a model call, because the alternatives would need so many rules and exceptions that they would be more complex than the model they avoid.

Solves means the option actually does the task, not that it handles the examples someone tried. That claim needs evidence, and gathering it is the subject of Chapter 2.

Well enough means there is a threshold, and it was set before the options were compared. Without one, every option can be made to look adequate or inadequate after the fact.

The principle cuts both ways. It argues **against** adding a model where a rule, a query or a form already solves the problem well enough, and **for** a model where the problem requires something rules cannot express. It doesn't decide anything by itself. It tells you which comparison to make.

Rules, learned models and execution behavior

Two distinctions matter here. **Rule-based versus learned** describes where a component's behavior comes from. **Deterministic versus stochastic** describes whether it produces the same output for the same input under the same state, configuration and other relevant conditions. These distinctions often appear together, but they are not the same.

A **deterministic component** produces the same output for the same input under the same state, configuration and other relevant conditions, regardless of how its behavior was obtained. A **stochastic component** may produce different outputs for the same input under otherwise equivalent conditions because its inference process samples from a distribution.

A **learned-model component** derives its behavior from a model trained on data rather than solely from rules someone wrote down. A language model is the main example in this book: at each step it defines a probability distribution over possible outputs, and the runtime either samples from that distribution or uses a decoding procedure that may be deterministic (Chapter 3). A learned model can therefore be deterministic or stochastic at inference time, depending on the model, decoding method and execution conditions. Its outputs are shaped by its input and may be highly consistent. Settings such as temperature can affect repeatability for the same input on a given setup (Chapter 3), but repeatability does not mean correctness or a complete specification: a wrong answer can repeat too, and you still cannot read off in advance what the component will do on an input nobody has tried.

Neither approach is better in general. Rule-based components tend to be predictable, cheap to run and straightforward to inspect, but they only do what their rules express. Learned models may handle inputs nobody anticipated — paraphrases, ambiguity, context, intent — but they can also fail on them. Whether they do is something you find out by measuring, not by reading the code. Whether their outputs repeat under fixed conditions is a separate property.

The difference that matters for engineering is **what you must do to trust each one**:

	Rule-based component	Learned-model component
Specification	The rules define the behavior	The model and its instructions guide behavior but do not fully specify it

	Rule-based component	Learned-model component
Testing	Cases can be enumerated from the rules	Behavior must be estimated from a sample of cases; tests show what happened on the cases tried
Failure	If deterministic, a failure repeats on the same input under the same conditions; its cause depends on the implementation and its fit to the task	May or may not repeat, depending on inference and execution conditions; can shift with wording or context; the cause may not be visible
Evidence of quality	Reading the rules, testing their cases and checking them on real inputs	Measuring on a representative set of cases (Chapters 10 and 11)
Residual uncertainty	Mostly about whether the rules match the problem, and how often real inputs fall outside them	About the rate and kind of errors, which must be estimated; output repeatability is a separate question

When you replace a rule-based component with a learned model, you change what the system can do and what you have to measure, test and observe. The model's behavior on a new input cannot be read from explicit rules. Its output may also be deterministic or stochastic under a specified setup; either way, its quality on new inputs must be established with evidence. That is often the right trade. It should always be a visible one.

This is a working engineering distinction, not a taxonomy every component fits cleanly. Real systems mix both kinds: a rule with a model fallback, a model constrained to a fixed set of outputs, a deterministic check on a model's output. The distinction is useful because it points to what must be measured.

Matching versus interpreting

A practical way to tell which kind of component a part of your system needs is to ask whether the task is **matching** or **interpreting**.

Matching asks whether an input contains, equals or satisfies something that can be written down: a word, a code, a format, a value in a range. Rules are good at matching.

Interpreting asks what an input *means*: what the user wants, whether two phrasings say the same thing, whether a passage supports a claim. That is a different problem, even when it looks like matching. Rules can approximate interpretation, often surprisingly well, for as long as the inputs stay close to what the rule's author had in mind.

The trouble starts when a rule is used to approximate interpretation and the inputs change: new features, new users or new kinds of content bring inputs the rule was not written for. The rule keeps matching exactly what it was written to match. What it matches no longer means what it used to. Nothing in the rule has changed, and nothing in the code signals the failure. The case study in this chapter is a clean example.

Complexity is a cost

Every option in a design comparison has costs; rules have to be maintained, and asking the user means more interface. A learned-model component brings costs that are easy to overlook, because the call itself is easy to write. It can add:

- **latency:** another call to wait for, often on every request;
- **cost:** compute or per-call charges;
- **operational dependencies:** a model to host and version, or a provider to depend on;
- **failure modes:** timeouts, unavailable models, malformed outputs, wrong outputs that look right;
- **observability requirements:** recording inputs, outputs and decisions so failures can be diagnosed;
- **evaluation requirements:** a way to know whether the component is good enough, and whether it got worse;
- **uncertainty:** errors that can't be enumerated from the code, at a rate that must be estimated.

None of these is a reason to avoid AI. They belong in the comparison next to the costs of the alternatives, before the decision is made, not discovered after the component ships. Chapter 12 treats latency and cost as measured design inputs, and Chapter 14 shows how to combine them with quality and constraints in one decision.

The engineering method

This book applies one loop to every decision:

Problem → Hypothesis → Baseline → Experiment → Measure → Compare → Decide → Observe → Iterate

In brief: state the **problem** and what "well enough" means; turn a belief about what will solve it into a **hypothesis** a specific observation could prove wrong; establish a **baseline** of how the system behaves now; run an **experiment** that changes one thing; **measure** what happened and under which conditions; **compare** the result with the baseline and the threshold, costs included; **decide**, and record why; **observe** the decision in use; and **iterate** when it no longer fits.

The loop matters for three reasons.

It puts the decision after the evidence. Implementing first and measuring afterwards turns every result into a justification for something already built.

It makes gaps visible. Real decisions often skip stages: no baseline, an experiment with one case, nobody observing. Named stages let you say which ones were skipped, which is the first step toward knowing how much a decision can be trusted.

It does not end. A decision made well for one problem can become wrong when the problem changes, without any change to the code.

This chapter establishes why the loop matters, and its case study shows the whole loop on a small decision, including the stages the decision went through only in part. How to run the middle of the loop is Chapter 2's subject; Chapter 14 returns to "decide".

The lab system

The evidence in this book comes from one system, called the **lab system** throughout. It is introduced here once. In later chapters it appears in Lab Notebook entries, not as the subject of chapters.

The lab system is a multi-tenant document question-answering platform. Users ask questions about their organization's documents. The system retrieves relevant passages from that tenant's documents, a language model generates an answer with citations, and a second model call checks whether the answer is supported by the passages. It runs local models on a CPU-only host, with an optional hosted-model fallback for cases the local path cannot answer. The prompts cited in this book are in this edition's wording, not transcribed from the originals.

The lab exists to provide **bounded, real engineering evidence**: decisions that were actually made, measurements that were actually taken, failures that actually happened, recorded in decision records and investigation records that the Lab Notebook entries cite as sources. It is not a reference architecture and not a recommendation. The technologies it happens to use appear only as context for its experiments, never as requirements.

Every lab result in this book is bounded by its conditions: one system, specific models and versions, one kind of hardware, specific task sets and sample sizes, often small. Evidence labels keep those bounds visible. This chapter uses three of the five labels defined in Chapter 2: *Observed in the lab*, *Not measured in this project* and *Illustrative*.

3. Options and Trade-offs

When part of a system has to decide something about its input — which path a request takes, which category a message belongs to, whether a document is relevant — there are several broad ways to build that decision. None is always right.

Option	What it buys	What it costs	Main risk
A. Fixed rules (keywords, patterns, lookup tables)	Usually predictable, fast and cheap; cases can be read from the rules and tested	Rules to maintain; cover only what they express	Gaps and overmatches can cause systematic failures: the same input fails the same way while the conditions stay the same
B. Ask the user (separate entry points, explicit choices, forms)	No inference needed; the user states the intent	More interface; users must know which option they want	Users choose wrongly or don't want to choose
C. Rules plus patches (add exceptions as failures appear)	Keeps the deterministic component; fixes the reported case	Exceptions to maintain; a patch written for one case leaves the rest of the class uncovered	The rule can grow harder to predict; the next failure in the class may not be prevented
D. A model-based classifier (a model call, or a small trained classifier)	Can interpret meaning, paraphrase and context	Latency, cost, dependencies, failure modes, evaluation	Errors that can't be enumerated from code, at a rate that must be estimated
E. Rules first, model for the rest	Model cost paid only on inputs the rules leave undecided	Two mechanisms to maintain and test, and a boundary between them	A rule that matches an input it was not written for still decides, and the model never receives that input

Each option is valid for some problem. **Fixed rules are often the right first answer**: a rule that is simple, fast and correct on the inputs the system actually receives is hard to beat. **Patching rules indefinitely** can feel cautious, but a patch for one case leaves the class of failure open, and accumulated exceptions can make the rule harder to maintain and predict. A change that covers the whole class is a different kind of fix, and it can be the right one. **A model-based classifier changes the evidence required**: "what will it decide?" stops being a question you answer by reading code and gives way to "how often is it right?", which you answer by measuring.

The table argues *for* starting with rules where they suffice, which does not mean recommending them everywhere. It argues *against* patching rules indefinitely, which does not mean rejecting rules. The choice depends on the problem, and the problem can change.

4. Lab Notebook

The entries below use the Lab Notebook format defined in Chapter 2. The lab's routing prompt and its two output words are described here, not transcribed; the words quoted from the prompt are translated.

A keyword rule and the problem that changed

> **LAB NOTEBOOK 1.1 — From keyword routing to model classification**
>
> **Evidence status** Observed in the lab
>
> **Source** Decision records for the single entry point with keyword routing (ADR 0006, including its later update) and for model-based routing (ADR 0024); image attachment on questions (ADR 0023); code history (commit `95df9af`, 2026-07-29)
>
> **Context** The lab system's interface originally had one input box for questions and another for diagrams; a recorded decision replaced them with a single entry point because users think about what they want to ask, not about which internal endpoint handles it. That entry point accepts either a question or a request for a diagram, and must decide which path each request takes. The first design used a fixed keyword list: the request text was checked, after removing accents, for words associated with requesting a visual (for example "diagram", "flow", "architecture", "chart" and "imagem", Portuguese for

"image"). Any match sent the request to diagram generation; otherwise it went to answer generation. Later, a separate change allowed users to attach an image to a question.

Metadata recorded / missing Decision records and code history recorded (keyword routing introduced 2026-07-24; model-based routing 2026-07-29). The classification model is not named in the record beyond "the resolved chat model". Runtime version and hardware not recorded in the source.

Question / Hypothesis Not a planned experiment. The question the incident raised was: is the keyword rule still sufficient for routing, now that users can attach images and ask about them?

Method A user's request failed. The failing question was reproduced with an attached test image, before and after the change.

Sample size One real incident; one reproduction of the failing question before and after the change. The change's commit separately reports three requests to the local stack with that question and an attached image, all routed to an answer; the sources consulted do not say whether those requests include the reproduction.

Result *Observed:* (1) The original record described routing as a plain substring check rather than an extra model call, and noted that it therefore added no latency and no extra failure mode; the later record (ADR 0024) describes it as deliberately chosen over a model call for zero added latency. The original record also predicted its weakness: a question that happened to contain a matching word would be routed to diagram generation even when a text answer was wanted. It accepted that trade-off for the first version and named the replacement to consider if false-positive routing became a real problem: a cheap classification prompt or a small local classifier. (2) After image attachment was added to the system, a user attached a photo and asked "O que tem nessa imagem?" ("What's in this image?"). The question contained "imagem", one of the keywords. The request was routed to diagram generation and returned a diagram built from unrelated retrieved documents instead of a description of the photo. (3) The later record quotes the user asking for the context to be interpreted rather than tied to fixed words, and it rejects removing only "imagem" from the list: that would have fixed the reported case and left the same class of failure for the next incidental match. The rejection is an argument, not a test. The keyword list was removed. Routing now makes a classification call at temperature 0 that asks the model to answer with exactly one of two words, one for diagram and one for answer. When an image is attached, the model is told only that one is attached, not what it shows. The prompt states that incidental words such as "image", "diagram" or "architecture" do not by themselves indicate a request for a diagram, and that questions about the content of an image or attachment should be answered. (4) In the reproduction, the same

> question with an attached test image produced a diagram before the change and a description of the image after it. (5) The single entry point now makes **one additional model call on every request**. The original single-purpose endpoints, which never routed through this check, were unaffected.
>
> **Interpretation** The keyword rule did not break. It kept matching exactly what it was written to match. The problem changed with the inputs: once users could attach images, the word “imagem” stopped indicating a request for a picture and started appearing in the most natural question about an attachment. The task had always been interpreting intent; the first design accepted matching words as an approximation. A new kind of input made the approximation fail, and the original record’s revisit condition was met.
>
> **Decision** Replace keyword routing with a model classification call, accepting one additional model call per request with the aim of routing on intent rather than on incidental words. The record says the routing call uses the same failure-handling path as the generation calls. The user-visible outcomes are not the same: a diagram-generation failure falls back to an empty diagram, while no fallback is documented for a routing failure (see Limitations). A regression test pins the exact failing question with the model’s verdict mocked.
>
> **Limitations** One incident and one reproduction; this is a recorded engineering decision, not a measurement. In the code of the change, the shared path covers the model call itself. A failed diagram-generation call falls back to an empty diagram; no fallback is documented for a failed routing call. The user-visible behavior after a routing failure was not verified. The regression test checks how the code handles the mocked verdict, not whether the model produces it. The classifier is not perfect: the decision record itself notes that it can misjudge an unusually ambiguous question. *Not measured in this project:* routing accuracy before or after the change, on any set of questions; the cost impact of the added call. No controlled measurement of its latency impact is recorded either.

Two things in this entry are easy to miss.

The first design was not a mistake. It used a simple technology with a trade-off accepted for the first version, and its record named both the predicted failure and what to consider if that failure became real.

What changed was the input, not the code. Image attachment was a separate feature with its own decision record, and nothing in it touched routing. But it changed the inputs routing received. Features interact through the inputs they create, not only through the code they share.

What the change left unknown

The decision traded one kind of uncertainty for another. The rule routed any request containing a listed word to diagram generation. That behavior was correct when the request actually asked for a diagram. The known failure class was narrower: a request seeking a different kind of answer could mention a listed word incidentally and still be routed to diagrams. The reproduction demonstrates one such case; the frequency of this class was not measured. The classifier may misjudge some questions, but which ones, and how often, is not visible from the code. The decision record calls this a qualitatively different risk. The change addressed the observed failure and routed the reproduced case correctly; the decision was accepted despite the measurement gap below.

> **LAB NOTEBOOK 1.2 — Routing accuracy (Not measured variant)**
>
> **Evidence status** Not measured in this project
>
> **Question** How accurately does each routing approach send requests to the intended path, and what does the change add in latency and in cost per request?
>
> **What would answer it** A labeled set of real requests covering both intents, including questions about attachments, diagram requests phrased without keywords, and questions that mention diagram-related words incidentally. Both approaches run on the same set, with accuracy reported per category and the routing step's latency and cost per request (compute for a local model, charges for a hosted one) measured under the same conditions.
>
> **What we have instead** One question from Notebook 1.1, exercised in the incident, its reproduction and the commit's three local requests, which may include the reproduction; the original record's prediction of the keyword rule's failure class; and a regression test for that question. Later code adds a timer for the routing step, and a code comment gives an approximate duration for the call, but the sources consulted record no sample, conditions or run behind that figure. Chapter 5 (Notebook 5.3) records the same accuracy gap.
>
> **Risk of deciding without it** The decision rests on the nature of the failure, a guaranteed misroute on a natural question, rather than on a measured improvement. The evidence does not show that the classifier routes better overall, how often it misroutes, or what the call adds in latency and cost. No check described here would detect a genuine diagram request sent to the answer path.

Many real decisions are made this way: a single clear failure, a fix that addresses its cause, and measurement that never happens because the visible problem went away. Chapters 10 and 12 show what the measurement would involve.

The case through the loop

Stage	What happened	How complete
Problem	One entry point for two kinds of request; route each correctly	Stated in the first decision record; no threshold described in this chapter
Hypothesis	The first design: keywords are sufficiently specific to requesting a visual. Later: a classifier will route on intent where words mislead	The first stated, with its predicted failure; the later one implicit in the decision
Baseline	The keyword rule's behavior on the failing question	One case, not a rate
Experiment	The failing question reproduced with an attached image, before and after	One case
Measure	Diagram before, description after	Observed, not measured across a set
Compare	The known failure class against an unmeasured error rate; one added call per request	Qualitative; costs not measured
Decide	Replace the rule; accept the added call	Recorded, with the trade-off stated

Stage	What happened	How complete
Observe	The first design's revisit condition was met when a user's request was misrouted	A user found the failure, and it matched the recorded condition; the classifier's behavior in use is not described
Iterate	The first design was revisited and replaced, and a regression test added	One iteration described; accuracy not tracked

The table is not a criticism of the first design or of its replacement. Some stages are strong, some rest on a single case, and none measures routing across a set of requests. Knowing which is which is what lets you say how far each design can be trusted.

5. Decision Guide

Before introducing an AI component

1. What is the problem, stated without naming a technology?
2. What threshold defines "well enough", and what latency and cost can you afford?
3. What must always be true?
4. What is the simplest option that might clear the threshold?
5. Does the task require matching or interpreting?
6. What would the AI component add (Section 2, "Complexity is a cost")?
7. What evidence will tell you it works?

Signs that a deterministic rule is no longer enough

- The rule matches words to infer meaning, and the words have started to appear for other reasons.
- A new feature has changed the inputs the rule receives.
- Each failure is fixed by adding an exception.
- The rule fails every time on a common, natural input.

- The revisit condition you recorded has been met.

When adding a learned-model component, record

- what it buys: the failure class it is meant to remove or the capability it is meant to add;
- what it costs, measured or explicitly marked as not measured;
- what becomes uncertain, and which errors would go unnoticed;
- what happens when it is unavailable or returns something unusable;
- what evidence you have, and with which label;
- what would make you revisit the decision.

6. Limits

This chapter establishes a way of thinking. Its evidence is deliberately small.

One observed case, not a benchmark. The case study is a single real failure and a single reproduction. It does not show that classifiers route better than keyword rules in general, or in this system overall. Nor do the records report an experiment comparing the classifier with other ways of routing, whether the separate input boxes the interface once offered or options they do not discuss, such as a fixed rule based on the presence of an attachment, or rules combined with classification. The case therefore cannot say whether the classifier was the simplest option that would have solved the problem well enough. The only general quantitative statement it supports is architectural: one additional model call per request. Routing accuracy and cost impact are *not measured in this project*, and no controlled measurement of the latency impact is recorded (Notebook 1.2).

One system, one language setting. The lab's keyword list mixed Portuguese and English words, and the failing question was in Portuguese. Whether the same rule would have failed, or the same classifier would have worked, in another language or another system is not known.

The principle is not a formula. "The simplest technology that solves the problem well enough" tells you which comparison to make. It does not make the comparison, set the threshold or measure the options. Chapters 2, 10 and 14 cover those steps.

7. Going Further

This chapter argued why an AI engineer should think this way. It did not show how to produce the evidence its own case study lacks: a baseline worth comparing against, an experiment that could refute a fix, a measurement that changes a decision.

Chapter 2 — Evidence Before Opinion: Running an Engineering Experiment shows how. It turns the middle of the loop — hypothesis, baseline, experiment, measure, compare — into a practical method, introduces the five evidence labels and the Lab Notebook format, and uses the lab system to show three proposed fixes measured before any was implemented.

The chapters after it apply the same stance to each part of an AI system:

- **Part II — Models** (Chapters 3 and 4): what a language model gives you and what it doesn't, and how to choose one.
- **Part III — Shaping Behavior and Knowledge** (Chapters 5 to 8): prompting, embeddings and retrieval, RAG as an engineering decision, and context engineering. Chapter 5 returns to the routing prompt from this chapter's case study.
- **Part IV — Systems That Act** (Chapter 9): tool calling and first agents.
- **Part V — Knowing Whether It Works** (Chapters 10 and 11): evaluating AI systems, and deciding when a benchmark deserves trust.
- **Part VI — Trade-offs** (Chapters 12 to 14): latency and cost as design inputs, security fundamentals, and making the decision.

Book 2 — Production AI Engineering covers running AI systems in production. **Book 3 — AI Model Lab** covers models and evaluation science in depth.